

COMP 10066

Week 10 Lecture

Website Testing Lecture #1

A Look at Web Usage

JUL
2024

ESSENTIAL DIGITAL HEADLINES

OVERVIEW OF THE ADOPTION AND USE OF CONNECTED DEVICES AND SERVICES



TOTAL
POPULATION



we
are
social

8.12
BILLION

URBANISATION

57.9%

UNIQUE MOBILE
PHONE SUBSCRIBERS



Meltwater

5.68
BILLION

vs. POPULATION

70.0%

INDIVIDUALS USING
THE INTERNET



KEPIOS

5.45
BILLION

vs. POPULATION

67.1%

SOCIAL MEDIA
USER IDENTITIES



KEPIOS

5.17
BILLION

vs. POPULATION

63.7%

Website Testing – Why?

1. Web sites have a potential large audience
2. They are public
3. Quality of site reflect on the company
4. Need to be professional
5. Can have a diverse set of users / uses

Website Testing – What?

- There are many components to a web site to be tested
 - Content – content is unique and correct, error free including spelling and grammar
 - Search Engine Optimized – website has keywords to ensure it shows in search engines when appropriate
 - Readability – is the content readable by the intended audience
 - Functionality – do all links, buttons, forms work

Website Testing – What?

- There are many components to a web site to be tested
 - Accessibility – Does it meet standards? Does it work regardless of device?
 - Usability – Is it “easy to use”?
 - Performance – Website is responsive, can handle expected load

*The points on this slide will be covered in Week 11 and Week 12 Lectures

Website Testing

- We will now discuss each of these components that need to be tested investigating the following:
 - Is the testing subjective or objective?
 - What testing techniques can you think of?
 - Is it possible to automate this testing?

Content

- How do we ensure the website content is correct and error free?
- Testing content is subjective or objective?
- Are there tools we can use?
- Can we automate this process in anyway?

Content

- Subjective in nature – depends on what your website is on
- Content needs to be proof read
 - No possible automation for actual content
 - An automated spelling / grammar checker can be used for errors
 - May require a domain subject expert

Content / Search Engine Optimized

- In order for a website to be successful, it needs to be found
- Many use Google's Analytics to decide if their website is good
- <https://www.reliablesoft.net/what-makes-a-good-website/#element1>
- Can also use site tuning to improve SEO, one example: <https://www.semrush.com/>

Readability

- How do we ensure the website content is readable?
What is considered readable?
- Testing readability is subjective or objective?
- Are there tools we can use?
- Can we automate this process in anyway?

Readability

- Depends on audience
 - General websites need to be readable for the general public
 - Specific audience websites can expect some sort of background knowledge
- Some automation is available to understand level
 - readable.io

Functionality

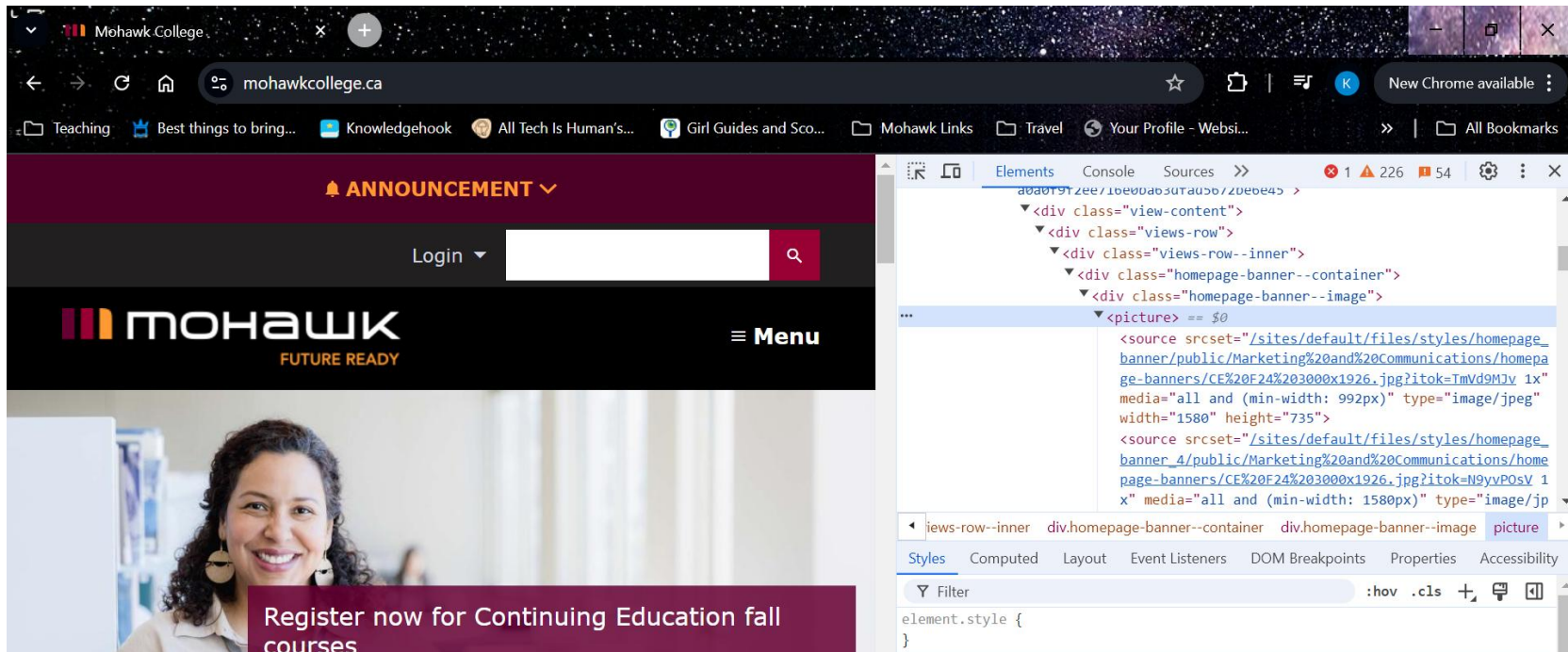
- How do we ensure the website content is functional?
- Testing functionality is subjective or objective?
- Are there tools we can use?
- Can we automate this process in anyway?

Functionality

- Objective – links, buttons, forms either work or they don't
- Tedious to test manually
- Tools exist to create scripts for regression testing to be run on each new release
 - Ensure all functionality is working
 - Like all testing, time needs to be invested to create and maintain scripts
 - In this course we will look at Selenium Web Driver tools

Functional Web Site Testing – How?

- What is Gray box-testing?
 - Partial view into the code on a web-page
 - HTML is visible some CSS/javascript may also be visible using Inspect tool in web browser



Functional Web Site Testing – How?

- What types of items should be **reviewed** in a gray-box test?
 - Use of standard / modern tagging.
 - Responsive design implemented for mobile devices
 - Use of outdated features
 - These are not always well supported in new devices
 - Information that should not be in production code
 - Comments

Selenium

- <https://www.selenium.dev/>
- Selenium IDE
 - Tool that integrates into Chrome, FireFox and Edge to create scripts for testing
 - Simple record and playback functionality
- Selenium WebDriver
 - API / Library to be used with a programming language

Selenium IDE Overview

Project: COMP 10066 Lecture Example*

Executing ▾

https://www.mohawkcollege.ca

	Command	Target	Value
1	✓ open	/programs/technology/computer-systems-technology-software-development-559	
2	✓ verify text	css=title > .field	Computer Systems Technology - Software Development - 559
3	✓ click	css=section-content	
4	✓ click	css=vertical-tabs__menu-item:nth-child(5).vertical-tabs__menu-item-title	
5	✓ verify text	css=field--name-field-course-overview > .field__label	Course Overview & Descriptions

Test Cases

Command: verify text

Target: css=field--name-field-course-overview > .field__label

Value: Course Overview & Descriptions

Description:

Runs: 1 Failures: 0

Log Reference

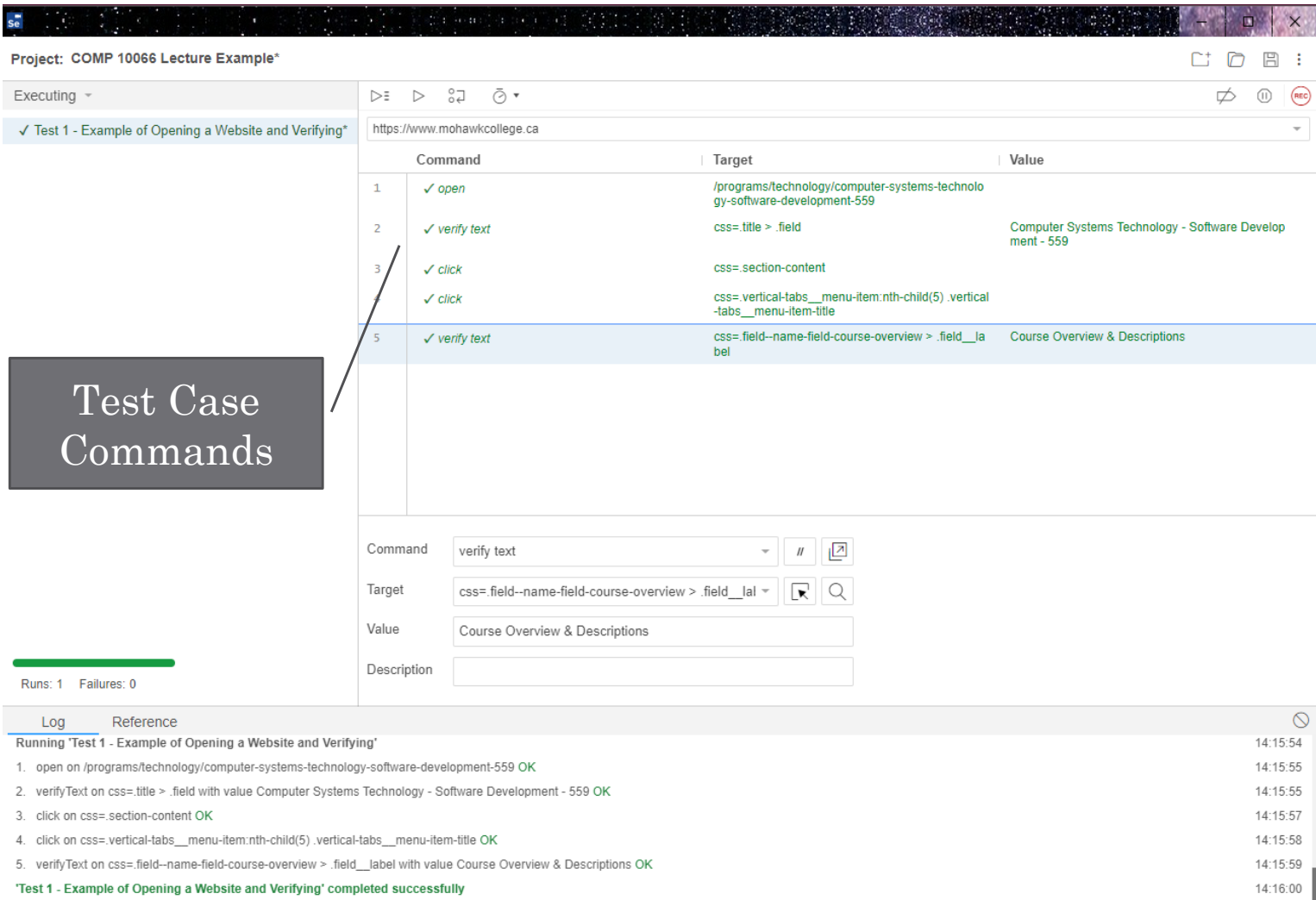
Running 'Test 1 - Example of Opening a Website and Verifying'

- 1. open on /programs/technology/computer-systems-technology-software-development-559 OK 14:15:54
- 2. verifyText on css=title > .field with value Computer Systems Technology - Software Development - 559 OK 14:15:55
- 3. click on css=section-content OK 14:15:57
- 4. click on css=vertical-tabs__menu-item:nth-child(5).vertical-tabs__menu-item-title OK 14:15:58
- 5. verifyText on css=field--name-field-course-overview > .field__label with value Course Overview & Descriptions OK 14:15:59

'Test 1 - Example of Opening a Website and Verifying' completed successfully 14:16:00

Selenium IDE Overview

Test Case Commands



Project: COMP 10066 Lecture Example*

Executing ▾

✓ Test 1 - Example of Opening a Website and Verifying*

https://www.mohawkcollege.ca

	Command	Target	Value
1	✓ open	/programs/technology/computer-systems-technology-software-development-559	
2	✓ verify text	css=title > .field	Computer Systems Technology - Software Development - 559
3	✓ click	css=section-content	
4	✓ click	css=vertical-tabs__menu-item:nth-child(5).vertical-tabs__menu-item-title	
5	✓ verify text	css=field--name-field-course-overview > .field__label	Course Overview & Descriptions

Command: verify text

Target: css=field--name-field-course-overview > .field__label

Value: Course Overview & Descriptions

Description:

Runs: 1 Failures: 0

Log Reference

Running 'Test 1 - Example of Opening a Website and Verifying'

- 1. open on /programs/technology/computer-systems-technology-software-development-559 OK 14:15:54
- 2. verifyText on css=title > .field with value Computer Systems Technology - Software Development - 559 OK 14:15:55
- 3. click on css=section-content OK 14:15:57
- 4. click on css=vertical-tabs__menu-item:nth-child(5).vertical-tabs__menu-item-title OK 14:15:58
- 5. verifyText on css=field--name-field-course-overview > .field__label with value Course Overview & Descriptions OK 14:15:59

'Test 1 - Example of Opening a Website and Verifying' completed successfully 14:16:00

Selenium IDE Overview

The screenshot displays the Selenium IDE interface. At the top, the project name is "Project: COMP 10066 Lecture Example*". The main area shows a test case titled "✓ Test 1 - Example of Opening a Website and Verifying*" with a URL of "https://www.mohawkcollege.ca". Below this, a table lists the commands and their targets:

	Command	Target	Value
1	✓ open	/programs/technology/computer-systems-technology-software-development-559	
2	✓ verify text	css= title > .field	Computer Systems Technology - Software Development
3	✓ click	css= section-content	
4	✓ click	css= vertical-tabs__menu-item:nth-child(5) .vertical-tabs__menu-item-title	
5	✓ verify text	css= field--name-field-course-overview > .field__label	Course Overview & Descriptions

A callout box with the text "Record Test Case Button" points to the "REC" button in the top right corner of the interface.

Below the table, the "Command" field is set to "verify text", the "Target" is "css= field--name-field-course-overview > .field__label", and the "Value" is "Course Overview & Descriptions".

At the bottom left, a green progress bar indicates "Runs: 1 Failures: 0".

The bottom section shows the "Log" tab with the following entries:

- Running 'Test 1 - Example of Opening a Website and Verifying'
- 1. open on /programs/technology/computer-systems-technology-software-development-559 OK
- 2. verifyText on css= title > .field with value Computer Systems Technology - Software Development - 559 OK
- 3. click on css= section-content OK
- 4. click on css= vertical-tabs__menu-item:nth-child(5) .vertical-tabs__menu-item-title OK
- 5. verifyText on css= field--name-field-course-overview > .field__label with value Course Overview & Descriptions OK
- 'Test 1 - Example of Opening a Website and Verifying' completed successfully

Selenium IDE Overview

The screenshot displays the Selenium IDE interface. At the top, the project name is "Project: COMP 10066 Lecture Example*". Below this, a toolbar contains icons for running, debugging, and other actions. The main area shows a list of commands for a test case titled "Test 1 - Example of Opening a Website and Verifying*". The commands are:

Command	Target	Value
1. ✓ open	/programs/technology/computer-systems-technology-software-development-559	
2. ✓ verify text	css=title > .field	Computer Systems Technology - Software Development - 559
3. ✓ click	css=section-content	
4. ✓ click	css=vertical-tabs__menu-item:nth-child(5) .vertical-tabs__menu-item-title	
5. ✓ verify text	css=field--name-field-course-overview > .field__label	Course Overview & Descriptions

A callout box with the text "Control Buttons to Run Test Cases" points to the toolbar. Below the command list, there is a detailed view of the selected command (verify text) with fields for Command, Target, Value, and Description.

Runs: 1 Failures: 0

Log Reference

Running 'Test 1 - Example of Opening a Website and Verifying'

Log	Reference
1. open on /programs/technology/computer-systems-technology-software-development-559 OK	14:15:54
2. verifyText on css=title > .field with value Computer Systems Technology - Software Development - 559 OK	14:15:55
3. click on css=section-content OK	14:15:57
4. click on css=vertical-tabs__menu-item:nth-child(5) .vertical-tabs__menu-item-title OK	14:15:58
5. verifyText on css=field--name-field-course-overview > .field__label with value Course Overview & Descriptions OK	14:15:59
'Test 1 - Example of Opening a Website and Verifying' completed successfully	14:16:00

Selenium IDE Overview

The screenshot displays the Selenium IDE interface. At the top, the project name is "Project: COMP 10066 Lecture Example*". Below this, the status is "Executing". The URL bar shows "https://www.mohawkcollege.ca".

A table lists the commands executed during the test:

	Command	Target	Value
1	✓ open	/programs/technology/computer-systems-technology-software-development-559	
2	✓ verify text	css= title > .field	Computer Systems Technology - Software Development - 559
3	✓ click	css= section-content	
4	✓ click	css= vertical-tabs__menu-item:nth-child(5) .vertical-tabs__menu-item-title	
5	✓ verify text	css= field--name-field-course-overview > .field__label	Course Overview & Descriptions

A grey box labeled "Output Window" is overlaid on the right side of the table. Below the table, the "Command" field is set to "verify text", the "Target" is "css= field--name-field-course-overview > .field__label", and the "Value" is "Course Overview & Descriptions".

At the bottom, the "Log" tab shows the execution results:

```
Running 'Test 1 - Example of Opening a Website and Verifying'
1. open on /programs/technology/computer-systems-technology-software-development-559 OK
2. verifyText on css= title > .field with value Computer Systems Technology - Software Development - 559 OK
3. click on css= section-content OK
4. click on css= vertical-tabs__menu-item:nth-child(5) .vertical-tabs__menu-item-title OK
5. verifyText on css= field--name-field-course-overview > .field__label with value Course Overview & Descriptions OK
'Test 1 - Example of Opening a Website and Verifying' completed successfully
```

Selenium IDE Documentation

- Lab Activity 7 in Week 11 will go over a complete walkthrough of how to install and use the Selenium IDE
- Selenium Documentation can be found here
 - <https://www.selenium.dev/selenium-ide/docs/en/introduction/getting-started>

Selenium Test Case Requirements

- Each test case you write in Selenium should be completely independent of all other test cases
- Each test case should do the following:
 1. Open the website under test – Selenium does this automatically by requiring the website at the top of the test case. Other tools you have to perform this action separately.
 2. Perform the steps / action under test.
 3. Verify the result. Using a verify command, verify that the action performed in step 2 was successful.
 4. End with the close command to close the browser, so you don't have too many windows open

Why independent test cases?

- Test cases can be run individually or as an entire suite
- If a test case fails, the others still have the ability to succeed (regression testing)
- Can easily narrow down where the issue is based on what test case fails

Target Field

- The Selenium tool uses a target field to identify where on the page the tool should look
- The tool is a basic string comparer
- Functions by looking in the CSS for the target tags, grabs the text from that tag and then compares it to the value provided in the last column

https://www.mohawkcollege.ca

	Command	Target	Value
1	<i>open</i>	/programs/technology/computer-systems-technology-software-development-559	
2	<i>verify text</i>	css=.title > .field	Computer Systems Technology - Software Development - 559
3	<i>click</i>	css=.section-content	
4	<i>click</i>	css=.vertical-tabs__menu-item:nth-child(5) .vertical-tabs__menu-item-title	
5	<i>verify text</i>	css=.field--name-field-course-overview > .field__label	Course Overview & Descriptions

Target Example

- Look at the Test 1 – Example of Opening a Website and Verifying
- Run the test case in Selenium, step 6 should fail due to timeout
- Notice the target for step 6:
`css=.odd:nth-child(18) > td:nth-child(1)`
- This is looking for the 18th odd tag on the page. Then it is looking for the 1st td tag inside
- Inspect the page and see if you can find this on your own

Target Example

- During your inspection you should have found a large number of tags labeled as odd
- The tags are also buried deep in the XML
- This is what is causing the timeout, it is taking too long to search using the css target path

Target Example

- Working with tables are tricky in Selenium, often require hard coded row values
 - Any changes to the website make the test case no longer valid
 - Best design uses an identifier at each row, instead of relying on table row numbers
- We will now look at modifying the path to use XPath (using html tags)

XPath Overview

- XPath – XML Path Language
- XPath is similar to paths in computer file systems
- Good reference at W3 Schools:
 - https://www.w3schools.com/xml/xpath_intro.asp

XPath Components

- // - double slash - denotes a relative path
 - Selenium needs to search for this anywhere on the page
- / - single slash – denotes an absolute path
 - Selenium expects what follows to be the next tag, doesn't search
- [] – square brackets – used to identify attributes you are searching for
- @ - used to identify an attribute

XPath Example

```
<table class="members" width="700">
  <tbody>
    <tr>
      <td id="admin"></td>
      <td >0</td>
      <td>admin</td>
      <td>adminP6ss</td>
      <td>Y</td>
      <td>Y</td>
      <td>admin@admin.net</td>
    </tr>
  </tbody>
</table>
```

XPath Example

- Using the snippet from the last slide write the following:
 1. An absolute path that retrieves the user name “admin” exists
 2. A relative path that retrieves the user name “admin” exists

XPath Example

- Using the snippet from the last slide write the following:
 1. An absolute path that retrieves the user name “admin” exists
 - `//table[@class='members']/tbody/tr[2]/td[3]`
 2. A relative path that retrieves the user name “admin” exists
 - `//td[@id='admin']`
 - In this example we aren't grabbing the actual text, but using the fact the table has been designed with a hidden field to identify each row with the user name

Xpath Examples Continued

- See the lecture handout